

2025 Jan Oppe 1 - Set 2

Section 2 - problem 1

Merge two dictionaries and sum on conflicts

PROBLEM STATEMENT

Write a program to merge two dictionaries. If a key is present in both dictionaries, sum the values. Assume the values are numbers (int or float).

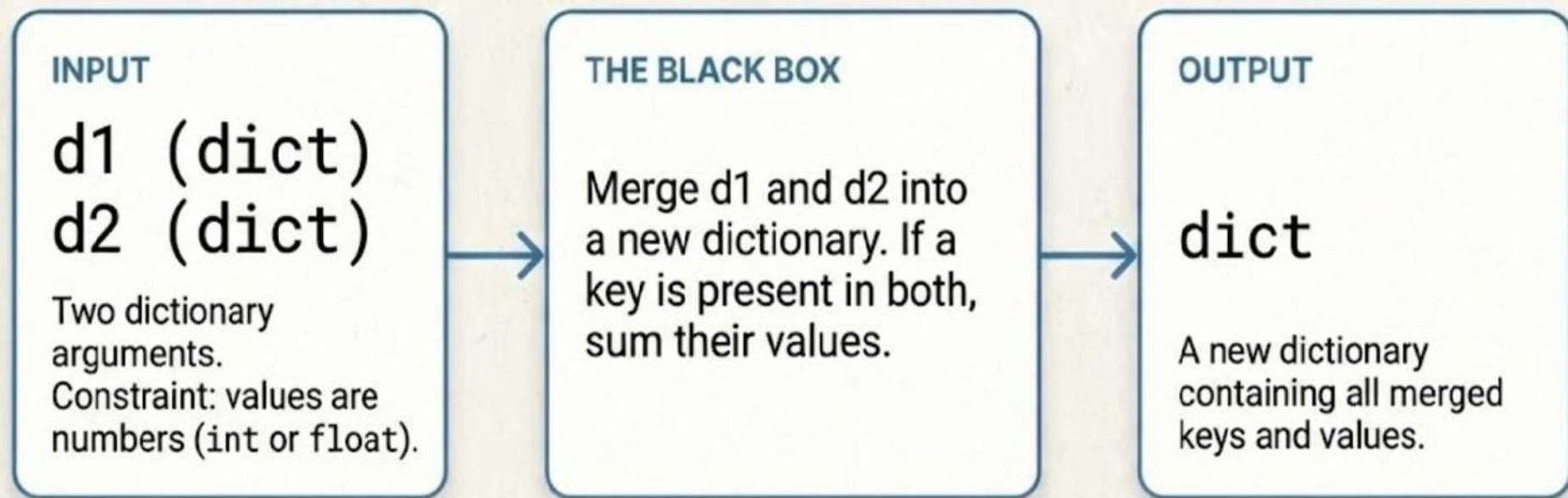


EXAMPLE & EXPLANATION

Given the inputs **d1 = {'a': 1, 'b': 2}** and **d2 = {'b': 3, 'c': 4}**:

- The result of **merge_dictionaries(d1, d2)** should be **{'a': 1, 'b': 5, 'c': 4}**.

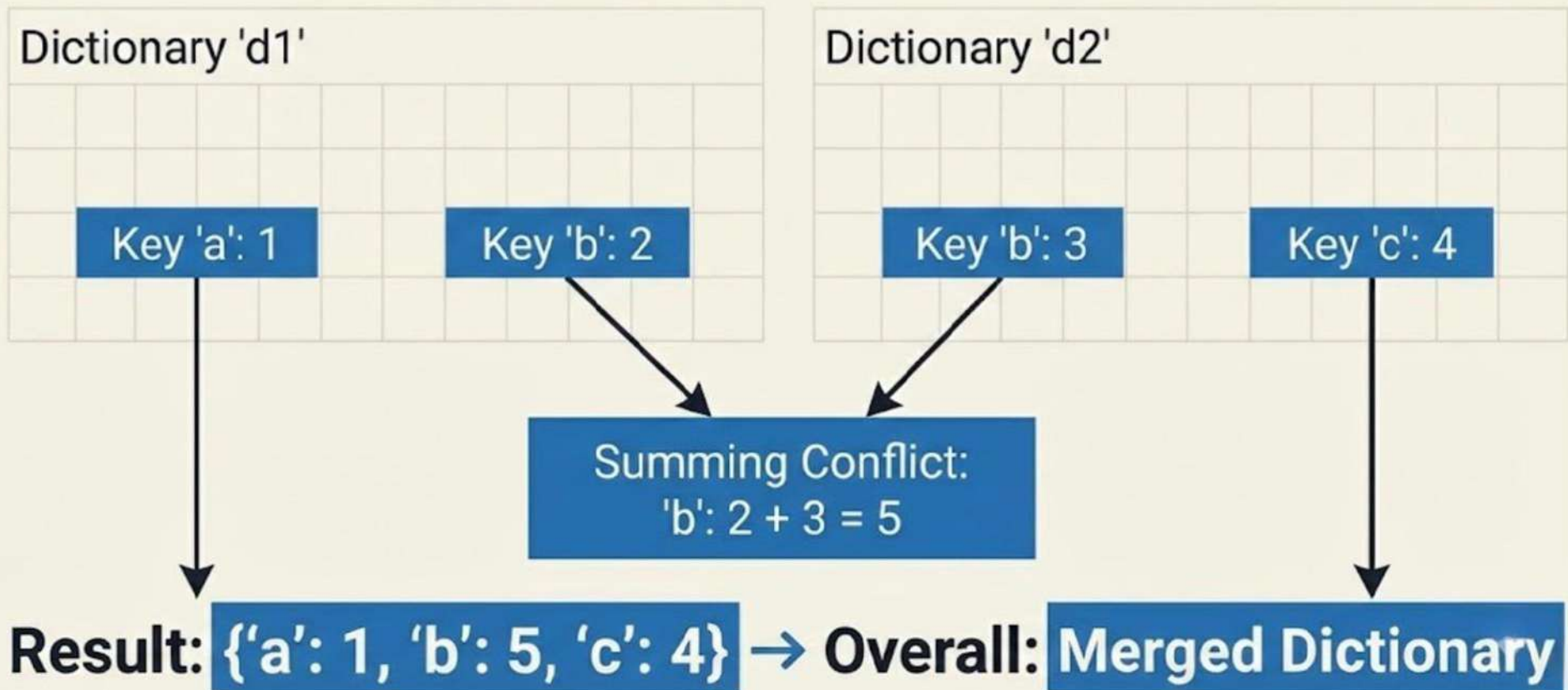
THE PROBLEM SPECIFICATION



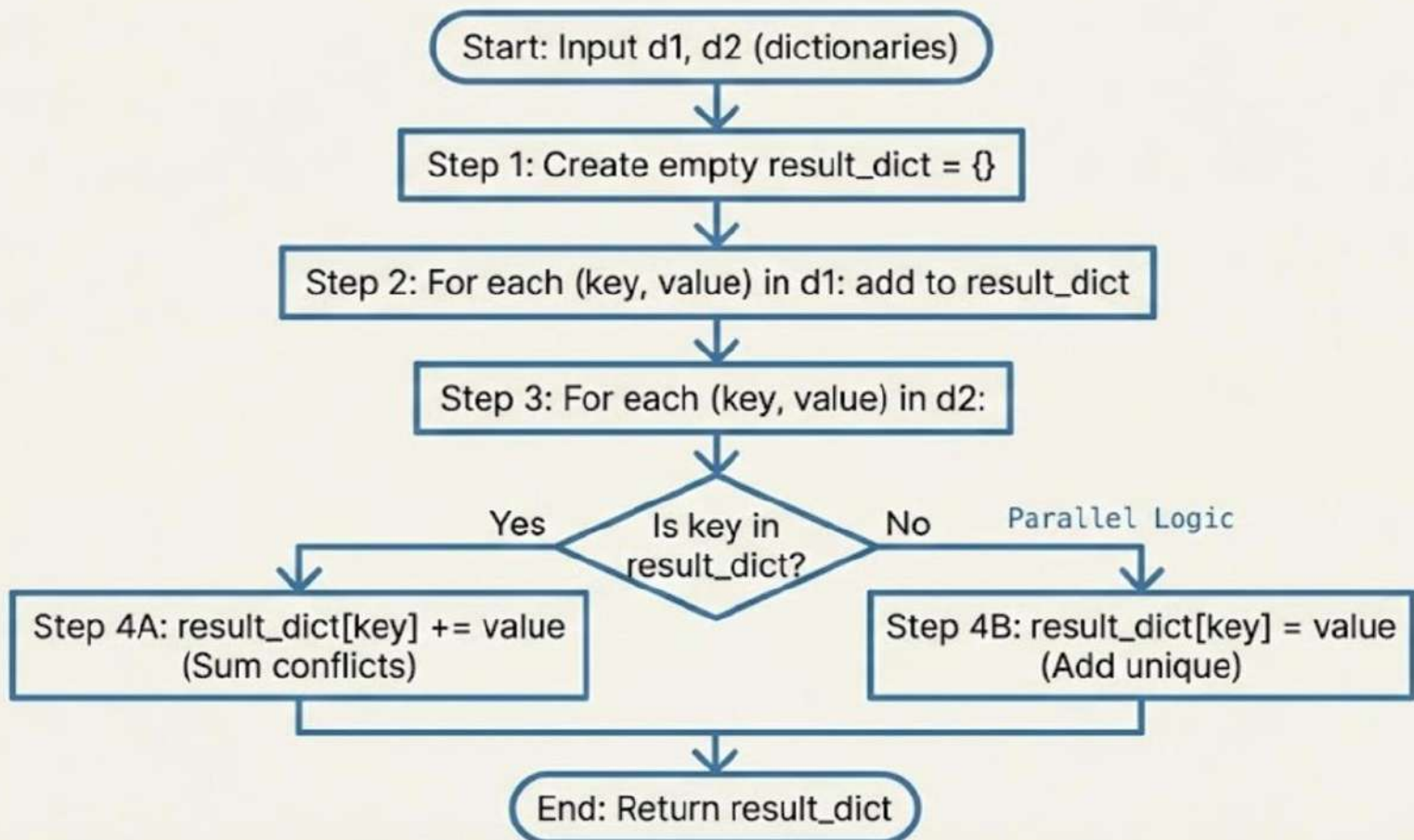
Note: This is a function type question, complete the required function definition.

Visualizing Dictionary Merge

Example Data: $d1 = \{ 'a': 1, 'b': 2 \}$, $d2 = \{ 'b': 3, 'c': 4 \}$



The Logic Flowchart



The Explanatory Approach

Step-by-Step implementation for clarity.

```
def merge_dictionaries(d1, d2):  
    # Step 1: Create a copy of the first dictionary  
    result_dict = d1.copy()   
  
    # Step 2: Iterate through the second dictionary  
    for key, value in d2.items():   
        # Step 3: Check for conflicts  
        if key in result_dict:   
            # Step 4a: Sum values for common keys  
            result_dict[key] = result_dict[key] + value   
        else:  
            # Step 4b: Add unique keys from d2  
            result_dict[key] = value   
  
    return result_dict 
```

Starts with all data from d1.

Goes through every item in d2.

Identifies keys present in both.

Sums values on conflict.

Adds new keys that are only in d2.

Returns the final merged dictionary.

The Pythonic Optimization

Leveraging built-ins for concise logic.

Type Hinting for clarity.

Addition Operator for merging & summing.

```
def merge_dictionaries(d1: dict, d2: dict) -> dict:  
    return dict(Counter(d1) + Counter(d2))
```

What is Counter?

A `Counter` is a specialized dictionary subclass from the `collections` module designed for counting hashable objects. Crucially, it supports mathematical operations like addition (`+`).

(`+`). When two `Counter` objects are added, they merge, and the values of any identical keys are automatically summed.

Concept Spotlight: Dictionary Keys

A dictionary's keys act as unique identifiers for accessing its corresponding values.

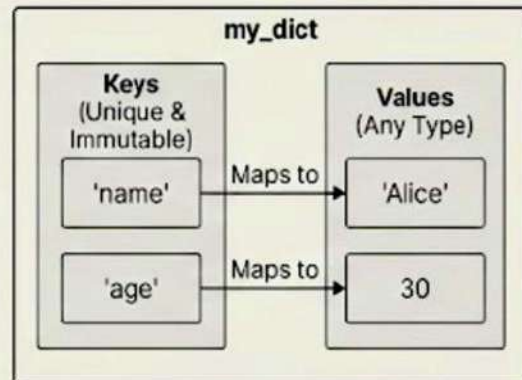
Explanation

In Python, a dictionary is a collection of key-value pairs. The keys must be unique and immutable (like strings, numbers, or tuples). You use the key to quickly retrieve, update, or delete its associated value, much like looking up a word in a physical dictionary.

Code View

```
my_dict = {'name': 'Alice', 'age': 30}  
print(my_dict['name']) # Output: Alice
```

Runtime Object



Application: Accessing user data. Input: `user_data['email']` -> Output: `"alice@example.com"`

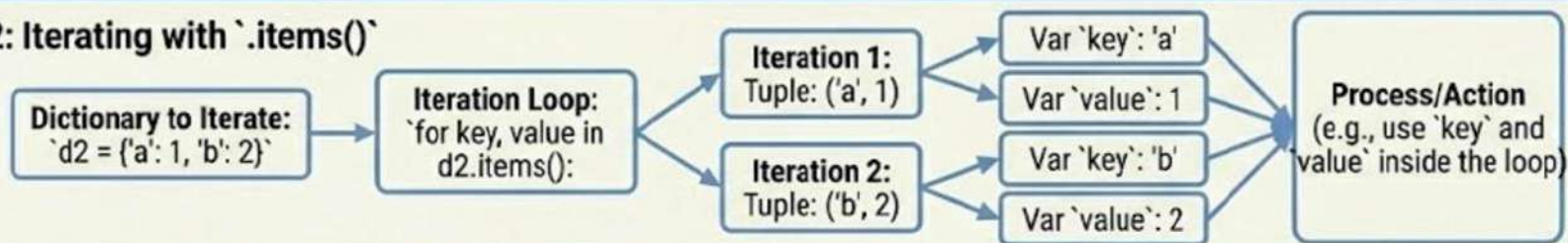
Concept Spotlight: Copying and Iterating Dictionaries

Understanding how to duplicate a dictionary and efficiently loop through its key-value pairs.

Part 1: Creating a Shallow Copy (`.copy()`)



Part 2: Iterating with `.items()`



Scenario A (Copying & Modifying)

Code: `d1 = {'x': 10}; d2 = d1.copy(); d2['x'] = 20`

Evaluation:
`d2` is modified to `{ 'x': 20 }`.
d1` remains `{ 'x': 10 }`.`

Outcome: `d1` and d2` are independent.`

Scenario B (Iterating & Printing)

Code: `d = {'y': 5}; for k, v in d.items(): print(f'{k}:{v}')`

Evaluation:
Loop runs once. `k` is y` and v` is 5`.`

Outcome: Prints `y:5`.`

Scenario C (Iterating & Summing)

Code: `d = {'p': 1, 'q': 2}; total = 0; for k, v in d.items(): total += v`

Evaluation:
Iteration 1: `total` becomes 1`.
Iteration 2: total` becomes 1+2=3`.
Outcome: total` is 3`.`

Robustness Rule: A shallow copy (`.copy()`) only duplicates the dictionary structure. If values are mutable (like lists), changes to those values will be reflected in both the original and the copy.

Solution Comparison



Manual Iteration (Long Solution)

- Iterates through the second dictionary's items one by one.
- For each key, manually checks its existence in the first dictionary.
- If the key exists, performs an explicit addition: `d1[key] = d1[key] + d2_value`.`



Pythonic Optimization (Using `Counter`)

- Converts both dictionaries into ``collections.Counter`` objects.
- Uses the single ``+`` operator, which automatically handles the summation logic.
- The result is a new ``Counter`` with summed values for all common keys.

Key Takeaways

01

Problem: Merge & Sum Dictionaries

Merge two dictionaries. If a key is present in both, sum their respective values.

02

Core Logic: Key Conflicts

Iterate through both dictionaries, identify common keys, and add their values. Unique keys are added as-is.

03

Return Value: New New Dictionary

The function returns a single dictionary containing all keys and the correct, summed values.

Final Syntax Cheat Sheet

```
import collections  
return dict(collections.Counter(d1) + collections.Counter(d2))
```